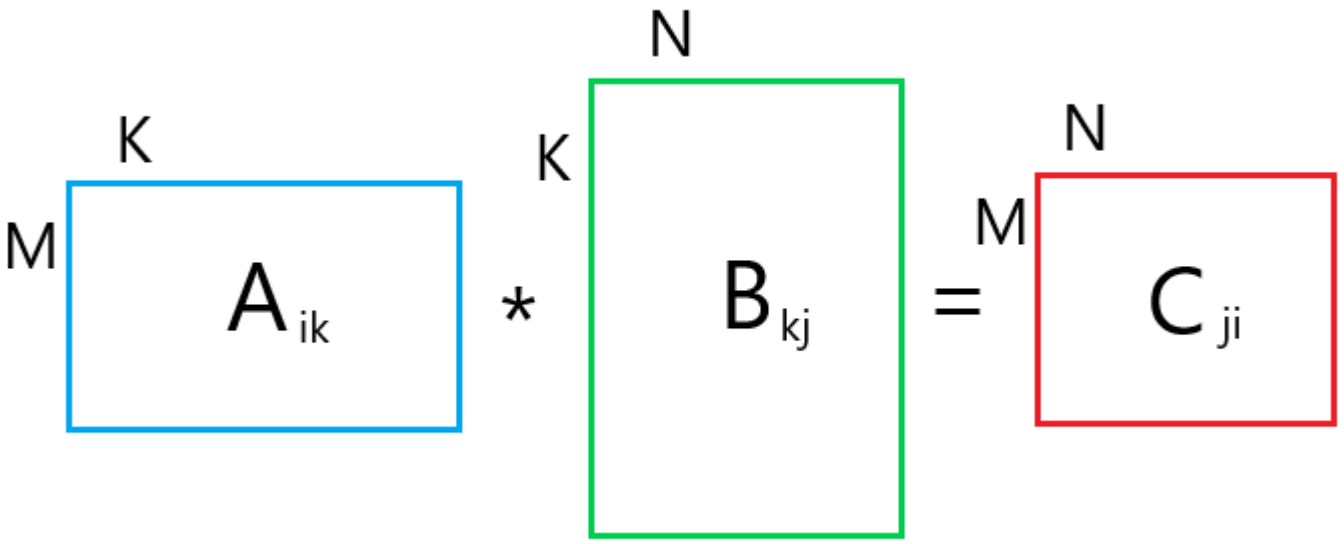
Описание алгоритмов minigemm версий 0 - 2 и конструкций их составляющих

Используемые обозначения

A — матрица размера M на K, B — K на N, C — M на N. Во всех вариантах алгоритма матрице C присваивается результат вычисления произведения $A \times B$.



Версия 0. Наивная реализация

Является переведённым на язык программирования Си математическим определением операции умножения матриц:

$$C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} B_{k,j}$$

Перевод “лобовой”, почти один-в-один, за исключением зануления матрицы C перед её заполнением.

Код на Си:

```
void gemm_v0(int M, int N, int K, const float * A, const float * B, float * C)
{
    for (int i = 0; i < M; ++i)
    {
        for (int j = 0; j < N; ++j)
        {
            C[i*N + j] = 0;
            for (int k = 0; k < K; ++k)
                C[i*N + j] += A[i*K + k] * B[k*N + j];
        }
    }
}
```

Версия 1. С изменённым порядком внутренних циклов

Для всех i от 0 до $M - 1$:

1. Заполнить i-ю строку матрицы C нулями.
2. Для всех k от 0 до $K - 1$:
    a. $a \leftarrow A_{i,k}$
    b. Для всех j от 0 до $N - 1$:
        i. $C_{i,j} \leftarrow C_{i,j} + a \times B_{k,j}$

Как видим, без $A_{i,k} \times B_{k,j}$ не обойтись. Но здесь переставлены два внутренних цикла.

Заполнение матрицы C теперь идёт построчно. Каждая строка обходится несколько раз, а именно K раз.

Кроме того, произведена небольшая оптимизация: часть вычисления адресов элементов матриц вынесена за циклы (пятая и десятая сверху строки).

Код на Си:

```
void gemm_v1(int M, int N, int K, const float * A, const float * B, float * C)
{
    for (int i = 0; i < M; ++i)
    {
        float * c = C + i * N;
        for (int j = 0; j < N; ++j)
            c[j] = 0;
        for (int k = 0; k < K; ++k)
        {
            const float * b = B + k * N;
            float a = A[i*K + k];
            for (int j = 0; j < N; ++j)
                c[j] += a * b[j];
        }
    }
}
```

Версия 2. С интринзиками (векторизованный внутренний цикл)

Получается из предыдущей версии алгоритма векторизацией операций во внутренних циклах. То есть вместо скалярных операций теперь используются интринзики, позволяющие обрабатывать по 8 скалярных значений float.

Код на Си:

```
void gemm_v2(int M, int N, int K, const float * A, const float * B, float * C)
{
    for (int i = 0; i < M; ++i)
    {
        float * c = C + i * N;
        for (int j = 0; j < N; j += 8)
            _mm256_storeu_ps(c + j + 0, _mm256_setzero_ps());
        for (int k = 0; k < K; ++k)
        {
            const float * b = B + k * N;
            __m256 a = _mm256_set1_ps(A[i*K + k]);
            for (int j = 0; j < N; j += 16)
            {
                _mm256_storeu_ps(c + j + 0, _mm256_fmadd_ps(a,
                    _mm256_loadu_ps(b + j + 0), _mm256_loadu_ps(c + j + 0)));
                _mm256_storeu_ps(c + j + 8, _mm256_fmadd_ps(a,
                    _mm256_loadu_ps(b + j + 8), _mm256_loadu_ps(c + j + 8)));
            }
        }
    }
}
```

Интринзики (Intrinsics)

1) __mm256 _mm256_setzero_ps(void);

Устанавливает все элементы 256-битного вектора float32 в ноль и возвращает этот вектор.

Неформально: возвращает вектор из 8 нулей типа float32.

2) __mm256 _mm256_set1_ps(float v);

Инициализирует 256-битный вектор скалярными значениями с плавающей точкой одинарной точности (float32), как указано в параметре float.

Неформально: возвращает вектор из 8 элементов, равных v.

3) __mm256 _mm256_loadu_ps(const float* a);

Загружает упакованные значения с плавающей точкой одинарной точности (float32) из 256-битной невыровненной области памяти, на которую указывает a, в вектор float32, который возвращается интринзиком.

Неформально: возвращает вектор из 8 элементов, полученных последовательно с указателя a.

4) void _mm256_storeu_ps(float* mem_addr, __mm256 a);

Выполняет операцию сохранения, перемещая упакованные значения с плавающей точкой одинарной точности (float32) из вектора float32 a в 256-битную невыровненную область памяти, на которую указывает mem_addr.

Неформально: записывает в 8 последовательных ячеек памяти, начиная с mem_addr, значения из вектора a.

5) __mm256 _mm256_fmadd_ps(__mm256 a, __mm256 b, __mm256 c);

Выполняет набор SIMD-вычислений умножения-сложения упакованных значений с плавающей точкой одинарной точности, используя три исходных вектора/операнда a, b и c. Соответствующие значения в двух операндах, a и b, умножаются, а результаты промежуточных вычислений с бесконечной точностью (т.е. без промежуточного округления после умножения) добавляются к соответствующим значениям в третьем операнде c, после чего окончательные результаты округляются до ближайших значений float32.

*Неформально: возвращает вектор a * b + c, где a, b, c – векторы из 8 элементов float32.*